API Authentication Models Evaluation: Quick Guide

What This Repository Is

This project is a practical comparison of three API authentication approaches:

1. Session-based authentication
2. JWT (JSON Web Token) authentication
3. OAuth 2.0 (Authorization Code + PKCE)

It is designed for evaluation and dissertation evidence, not just app development.

What It Tries to Answer

The repository compares these models in a controlled environment to answer questions like:

1. Which model is more resilient against common attacks?
2. What breaks when common security misconfigurations are introduced?
3. How do the models compare on performance overhead?
4. How large and complex are secure vs misconfigured vs AI-generated implementations?

How It Works

The same backend stack is used for all models so comparisons are fair.

Core stack:

1. Node.js + Express + TypeScript
2. Prisma + PostgreSQL
3. Jest + Supertest for tests

The secure baseline implementation lives in the src folder. Tests and performance scripts run against this baseline.

The Three Evaluation Layers

1) Secure Baseline

The repository includes secure implementations for Sessions, JWT, and OAuth.

These are evaluated using:

1. Unit and integration tests
2. Attack-focused tests
3. Coverage and performance tests

2) Misconfiguration Variants

The repository intentionally introduces security mistakes in controlled variants (for example weak JWT validation, weak cookie settings, OAuth state/redirect issues).

Each variant is tested with focused exploit tests to prove the security regression is real and measurable.

3) AI-Generated Samples

The repository also contains AI-generated samples for each auth model.

These are evaluated as artifacts (code quality and security-pattern checks), then summarized in report files.

What It Compares

The project compares baseline, misconfigured, and AI-generated implementations across:

1. Security behavior (pass/fail against attack and focused exploit tests)
2. Performance (latency/throughput baseline vs attack conditions)
3. Code footprint and complexity (size, functions, cyclomatic complexity, maintainability)
4. Consistency and completeness of expected controls

Key Folders (Simple View)

1. src: secure baseline implementation
2. tests: attacks, functional tests, variants, performance
3. misconfigurations: targeted insecure overrides
4. ai-generated: generated samples and analysis results
5. docs: dissertation-facing summaries and evidence tables
6. scripts: report generation and orchestration utilities

Typical Evidence Workflow

Use this sequence to regenerate core evidence:

1. Baseline tests and coverage
2. Focused variant exploit tests
3. Variant differential reports
4. Performance analysis
5. AI sample analysis and reporting
6. Code footprint report

Before Opening a PR

Run the same checks CI expects:

1. npm run docs:generate
2. npm run docs:lint
3. npm test

The CI workflow runs docs and tests in parallel and uses a single quality-gate status for merge decisions.

What to Read First in docs

If you only read a few files, start here:

1. TEST_EVIDENCE_MATRIX.md
2. DISSERTATION_EVALUATION_TABLE.md
3. VARIANT_DIFFERENTIAL_REPORT.md
4. AI_EVALUATION_SUMMARY.md
5. CODE_FOOTPRINT_SUMMARY.md
6. UNIFIED_COMPARISON_MATRIX.md

One-Line Summary

This repository is a controlled, test-driven comparison of Sessions vs JWT vs OAuth, showing how secure baselines behave, how targeted misconfigurations fail, and how AI-generated alternatives differ in security quality, performance, and code complexity.